A DH-Transformation-Based Neural Network Architecture for Forward Kinematics Learning

Leandro de Souza Rosa[1], Luca Iocchi[1]

***Abstract*— Forward kinematics is arguably the most basic computation on robotics, in which the Cartesian position of a given joint is computed from the joints positions. While industrial robots realize this computation using precise models, generic or built-in-house robots require learning a forward kinematics model, which is done in the literature by the means of a large monolithic function approximator that directly maps joints positions into Cartesian coordinates for each joint. Despite the monolithic approach success, these models fail to capture the forward kinematics linked nature, in which the estimation of one joint is linked to the previous one's estimation. Towards improving the forward kinematics learning, this paper leverages the Denavit-Hartenberg transformations notation for constructing a novel Neural Network architecture that realizes linked forward kinematics estimations. The proposed architecture is shown to improve learning efficiency and accuracy when compared to the traditional monolithic architectures found in the literature using simulated robotic serial manipulators with up to 7 joints.**

I. Introduction

Forward kinematics is arguably one of the most basic computations in robotics, which provides means of computing the position of joints, or other links, in a chosen reference frame (often the world frame). This computation is commonly found in Cartesian impedance [1] and compliance [2] controllers, and also has seen recent application in collision detection [3]. For industrial and commercial robots, the forward kinematics is computed based on a detailed and accurate system description provided by the manufacturer.

Nevertheless, for low-cost or in-house-built robots, achieving a satisfactory forward kinematics is not a trivial task, and often learning-based methods are adopted [4].

In order to learn forward kinematics, common approaches found in the literature often rely on using a monolithic Neural Network (NN) to approximate the mapping between joints and position, without giving much attention to the NN architecture and its impacts in learning efficiency and final model precision. Furthermore, little attention has been given for the learning scalability as the number of joints increase.

In this paper we propose a novel NN architecture based on Denavit-Hartenberg (DH) transformations for learning forward kinematics. The proposed architecture consists of one NN for estimating the transformation of each joint with respect to the previous one, and leverages the chaining properties of Deep Learning (DL) transformations for computing the forward kinematics with respect to a common reference frame.

In order to test the proposed architecture, we simulate robotic manipulators with 2, up to 7 joints, showing increasing gains in sampling efficiency and accuracy as the number of joints increase. We further demonstrate the impacts of using the proposed architecture on higher level computations based on the forward kinematics, namely velocity estimation through Jacobian computation and on a Reinforcement Learning (RL) policy learning.

II. Related Works

Learning forward kinematics using a monolithic NN as function approximator has been explored since early works, E.g., in [5] the approach is used for a simple planar 2-link robotic arm. More recent efforts show that this approach can scale for robots with 6-Degrees of Freedom (DoFs) [6], and also demonstrate the model's applicability to real robots by transferring a model learned from simulation [7], [8]. Towards learning a forward kinematics model for real robots, [9] proposes a vision based method to learn the linkage between joints, and uses an ulterior Gaussian Process Regression (GPR) for learning a monolithic forward kinematics model for a 6-DoFs robot.

Recent works propose exploring the coupling between forward and inverse kinematics for simultaneously learning both functions. For example, a monolithic NN is used for learning the forward and inverse kinematics of 2-DoFs [10] and 3-DoFs [11] robotic arms. In [12] the forward and inverse kinematics together for 6-DoFs serial and parallel robots using a monolithic NN which is trained online using a Locally Weighted Projection Regression (LWPR). Further than the forward kinematics, [13] proposes learning forward dynamics and inverse kinematics model of serial robot manipulators using Generative Adversarial Networks (GANs).

Instead of using NNs as approximator, [14] evaluates different function methods for forward kinematic approximation learning for continuum robots, highlighting the potential of NNs, Support Vector Machines (SVMs), Radial Basis Functions (RBFs), and fuzzy systems according to the dataset size. Furthermore, [15] proposes using Support Vector Regression (SVR) for learning the forward kinematics of both serial and parallel manipulators.

Recent research has been leveraging other types of information for improving the forward kinematics leaning efficiency. As example, [16] proposes framing the forward kinematics learning of a 2-DoFs robot as an explainable NN, using prior knowledge of the systems dynamics to

[1]Department of Computer Engineering, Robotics and Management, Sapienza University of Rome, 00185 Rome, Italy leandro.desouzarosa@uniroma1.it, iocchi@diag.uniroma1.it

improve the learning efficiency and trustworthiness; and in [17] differential information is used to improve the training of a monolithic forward kinematics module for a 7-DoFs robotic arm.

Recent attention has been given to the representation and forward kinematics learning procedure. For example, [18] studies the joint-position representation influence on forward kinematics learning with a monolithic NN for a 6-DoFs robotic arm, showing that quaternions can achieve higher accuracy when compared to Euler angles for estimating orientation. And [19] studies the training procedure effects by evaluating different data sampling techniques and optimization options for learning a monolithic forward kinematics model for a 2-DoFs robot.

We would like to highlight that all approaches above mentioned use a monolithic NN for learning a forward kinematics model. Instead, in this paper we propose a joint-by-joint NN model which leverages DH transformations for composing a global forward kinematics, and as such, the improvements achieved are expected to reflect on those approaches.

Nevertheless, learning-based methods for forward-kinematics are still not viable regarding sample efficiency and real robots require fine-tailored parametric models for obtaining a forward kinematics approximation [20]. However, it is hard to define a parametric model without making any assumption on the joints types and rotation axis. In comparison, the approach proposed in this paper makes no such assumptions, and can be easily applied to any robot.

Furthermore, our approach share the philosophy of [21], where the forward kinematics are computed link-to-link for a 21-DoFs humanoid robot through the kinematic closed-form equations in order to obtain improve computation efficiency. Similarly, our proposed network learns a link-to-link forward kinematics, but leveraging a NN novel architecture which explores DH transformations.

III. Preliminaries

Consider the serial robot manipulator illustrated in Figure 1 with j links and $j+1$ joints. The reference frames of the world and of J_0 are fixed and coincide.

Our task is to estimate the Cartesian position of each joint $\mathrm{J}_i | i \in \{1, \ldots, j\}$ in the world reference frame ($\mathbf{x}$) as a function of the rotation angles of $\mathrm{J}_i | i \in \{0, \ldots, j-1\}$ ($q_i | i \in \{0, \ldots, j-1\}$), as described in Equation 1. The Cartesian pose of each joint J_i is a d dimensional array (usually $n = 2, 3$), resulting in the estimation of $n \times j$ functions in total.

$$\tilde{\mathbf{x}}(\mathbf{q}) = \left[\begin{array}{cccc} \tilde{\mathbf{x}}_1(q_0) & \tilde{\mathbf{x}}_2(q_0, q_1) & \ldots & \tilde{\mathbf{x}}_j(q_0, \ldots, q_{j-1}) \end{array} \right] \tag{1}$$

The function $\mathbf{x}(\mathbf{q})$ can be expressed using DH transformation matrices [22]. For a given J_i, the a transformation between its reference frame and the one of J_{i-1} is described by 2, where $\mathbf{T}$ and $\mathbf{D}$ are the rotation and displacement matrices, respectively.

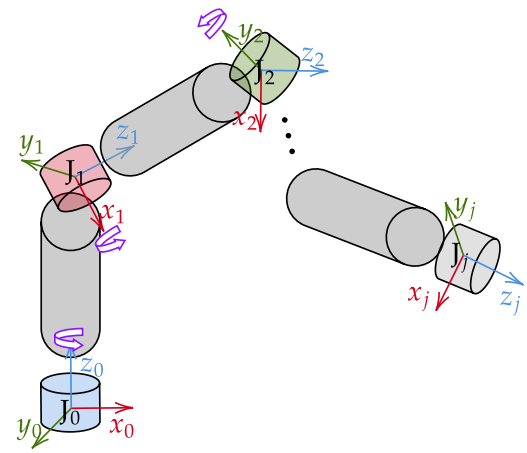

Fig. 1. A serial manipulator with j axes and $j+1$ joints J_i. Each axis (gray) is connected by a joint with a Cartesian reference frame (red, gree, blue) and can rotate over an arbitrary axis (purple). We consider, without loss of generality, the position of J_0 fixed at the world origin, and J_j to be the end-effector, which does not rotate. Elements are colored according to their rotation axis if they rotate, otherwise in gray. The Cartesian position of joint $\mathrm{J}_i | i \in \{1, \ldots, j\}$ ($\mathbf{x}_i$) in the reference frame of J_{i-1} depends only on q_{i-1}.

$$\mathbf{T}^i_{i-1} = \left[\begin{array}{cc} \mathbf{R}^i_{i-1}(q_{i-1}) & \mathbf{D}^i_{i-1}(q_{i-1}) \\ \mathbf{0}_{1\times 3} & 1 \end{array} \right] \tag{2}$$

For a serial manipulator, the transformations can be chained for obtaining a transformation for any given joint J_i in the world reference frame, according to Equation 3.

$$\mathbf{T}^i_0 = \prod_{k=1}^{i} \mathbf{T}^k_{k-1} \mid i \in \{1, \ldots, j\} \tag{3}$$

IV. Proposed Architecture

A. Overview

While approaches in the literature propose to learn a monolithic approximator $\tilde{\mathbf{x}}^m = \Phi^m(\mathbf{q}) : \mathbb{R}^j \rightarrow \mathbb{R}^{n \times j}$. Our main idea is to learn one approximator for each joint J_i, which encodes $\mathbf{T}^i_{i-1}$, and thus learning j predictors $\tilde{\mathbf{x}}^l_i = \Phi^l_i(\mathbf{q_i}) : \mathbb{R}^1 \rightarrow \mathbb{R}^{n \times (n+1)} \mid i \in \{1, \ldots j\}$, and joining all Φ^l_i leads to the proposed model Φ^l. Figure 2 presents the proposed architecture's overview.

We implement Φ as NNs with l_n fully-connected layers, and l_s neurons for each layer. Each layer, except for the last, is followed by a `TanH` activation layer. While other approximators could be used, e.g., Gausian Processes (GPs) as in [9], their evaluation falls out-of-the-scope of this paper.

B. Loss Function

For training Φ, we follow the approaches in the literature [6]–[11], [14], [16], [17], [19] and use the average Euclidean distance between the joints and their respective prediction, as described in Equation 4. Note that some approaches in the literature consider only the end-effector position and its prediction, which is reasonable since in a real robot, the position of individual joints is hard to obtain, while the end-effector's position can be obtained with off-the-shelf trackers

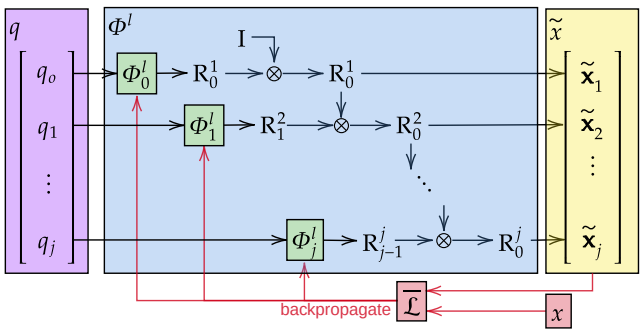

Fig. 2. Proposed architecture. For each J_i $i \in \{1, \dots, j\}$, a network Φ_i^l (green) learns $\mathbf{R}_{i-1}^i$, which is the DH transformation from J_{i-1} to J_i in function of q_i (purple). $\mathbf{R}_{i-1}^i$ is multiplied by the cumulative transformation of all previous joints, resulting in $\mathbf{R}_0^i$, from which the Cartesian position $\tilde{\mathbf{x}}_i$ (yellow) is obtained directly. Training is done using a loss $\mathcal{L}$ base on the Cartesian distance between the estimated and real values (red) as described in Section IV-B.

and cameras [9]. In contrast, since we perform the training in simulation, we have the flexibility of considering all joints, leading to a richer loss signal, improving the training convergence.

$$\mathcal{L} = \frac{1}{j} \sum_{k=1}^{j} ||\tilde{x}_k - \mathbf{x}_k|| \tag{4}$$

C. Training Procedure

Since in this stage we use simulation, both Φ^m and Φ^l are trained in a straight-forward fashion according to Algorithm 1. For each epoch of training, $nSamples$ random poses are sampled in joint space (line 3), and the respective Cartesian position of each joint is obtained directly from the simulator (line 4) and added to a mem memory buffer (line 5). The size of mem is limited, and upon additions further than its limit, it drops a percentage of its oldest data (we use a 10% data drop in all experiments) for reducing data allocation overheads. Next, $nIter$ iterations of training are done with the data within mem by randomly sampling bs samples from the memory (line 8), computing the average loss (lines 9 to 14), which is then back-propagated through the network (line 15) with an lr learning rate.

It is worthy to notice that in our tests, we observed that randomly sampling poses leads to a much higher efficiency than performing random walks, which is to be expected due to the better state space coverage. However, when considering a real robot in the RL loop, it is unpractical to sample the state space this way, since the robot will be naturally performing a random walk.

V. Results

A. Testing Environment

We build upon the `Reacher-v4` environment from OpenAI `Gym` [23], which consists of a 2D serial manipulator with 2 joints simulated using `MuJoCo` [24]. In order to evaluate the proposed approach for more realistic robots, we extend

Algorithm 1: Training procedure for Φ.

1 **for** $e \in \{1, \dots, epochs\}$ **do**
2 **for** $r \in \{1, \dots, nSamples\}$ **do**
3   $q \leftarrow$ random joint positions;
4   $x \leftarrow$ Cartesian position of all joints given q;
5   add (q, x) to mem;
6 **end**
7 **for** $t \in \{1, \dots, nIter\}$ **do**
8   $(qs, xs) \leftarrow$ get bs random samples from mem;
9   $\bar{\mathcal{L}} \leftarrow 0$;
10   **for** $(q, x) \in (qs, xs)$ **do**
11    $\bar{\mathcal{L}} \leftarrow \bar{\mathcal{L}} + \mathcal{L}$;
12   **end**
13 **end**
14 $\bar{\mathcal{L}} \leftarrow \bar{\mathcal{L}} / nIter$;
15 backpropagate($\bar{\mathcal{L}}$, lr) through Φ;
16 **end**

the environment enabling for 3D, and adding models for robots up to 7 joints[1].

The Φ modules are implemented using `PyTorch` [25], allowing for integration with `Gym`, and the derivatives used in Section V-C are computed using `PyTorch`'s `Autograd` [26].

For the sake of briefness we restrict the results presented in this section for the 3D case, as robots with more than 3 joints usually act in the three dimensional space.

B. Parameters Tuning

In order to assure appropriate hyper-parameters for Φ, we use `Ray Tune` [27] with `Optuna` [28] as search engine and a `Asynchronous Hyper-Band` [29] as scheduler. The hyper-parameters are sampled in the interval presented in Table I, We empirically found that $nSamples = 100$ and $nIter = 25$ are appropriated values tuning hyper-parameters for Algorithm 1.

TABLE I

Φ HYPER-PARAMETERS SEARCH SPACE. "LOG UNIFORM" SAMPLING CHOOSES A RANDOM NUMBER IN AN UNIFORM DISTRIBUTION IN THE LOG OF THE GIVEN RANGE, "CATEGORICAL" SAMPLING CHOOSES A RANDOM VALUE FROM THE RANGE LIST.

hyper-parameter	sampling	range
lr	log uniform	$[10^{-5}, 10^{-3}]$
l_n	categorical	$[2, 3, 4, 5, 6, 7, 8]$
l_s	categorical	$[8, 16, 32, 64, 128]$
bs	categorical	$[8, 16, 32, 64, 128]$

During the tuning process, the average loss over 10 learning trials $\bar{\mathcal{L}}^{tune}$, $epochs$, and the total number of parameters in Φ ($|\Phi|$) are passed to `Optuna` every 50 $epochs$ until $\bar{\mathcal{L}}^{tune} <= 0.005$ or $epochs = 2000$ are reached. Optimizing

[1]Publicly available source code: https://github.com/lsrosa/kinematics_learning

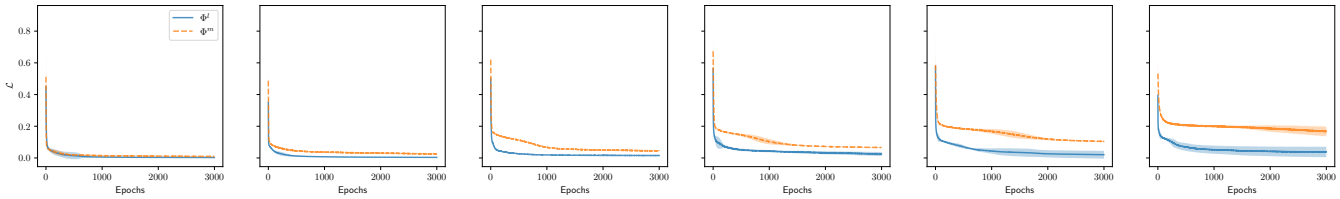

Fig. 3. Loss comparison between Φ^m and Φ^l. The solid line and shaded area represent the mean loss and the area between minimum and maximum losses over 30 trials.

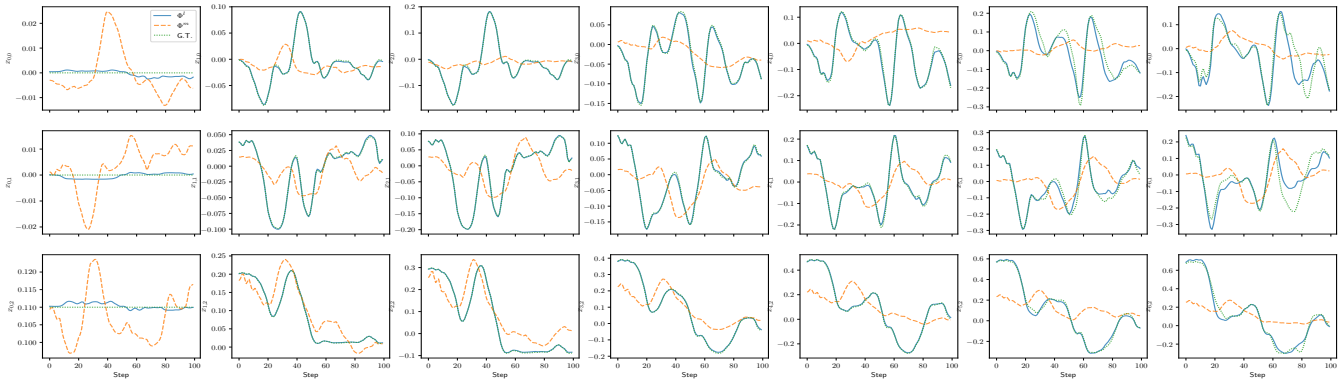

Fig. 4. Joints position estimation comparison for Φ^m and Φ^l for 100 steps.

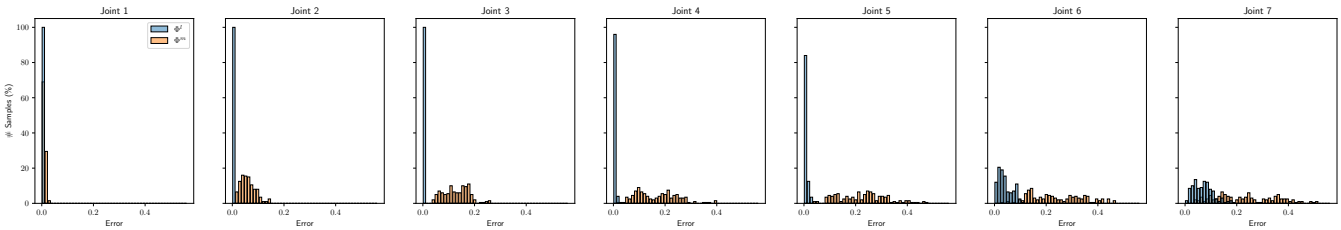

Fig. 5. Accuracy comparison for Φ^m and Φ^l.

$\bar{\mathcal{L}}^{tune}$ leads for hyper-parameters which allow for effective learning the forward kinematics; optimizing *epochs* incentivize the optimizer finding hyper-parameters that improve the learning sampling-efficiency; and optimizing $|\Phi|$ gives priority for smaller networks.

A total of 50 hyper-parameters samples are computed and collected by `Ray Tune`, and the final values are given by the sample with smallest $\bar{\mathcal{L}}^{tune}$ among the Pareto dominating samples according to $\bar{\mathcal{L}}^{tune}$, *epochs*, and $|\Phi|$. Table II presents the hyper-parameters found for Φ^m and Φ^l for robots with $j = \{2, \ldots, 7\}$. The found hyper-parameters values demonstrate the consequences of using a monolithic architecture: the complexity of the function being approximated grows with j (as described in Equation 1), and hence a larger NN is more suitable. In comparison, the function being approximated with the proposed architecture is constant in j, and increasing j increases the number of approximators linearly. As result, the proposed architecture is expected to scale better with j than its monolithic counterpart, both in precision and sample efficiency.

TABLE II

HYPER-PARAMETERS FOUND FOR ROBOTS WITH VARYING NUMBER OF JOINTS.

Φ^m						
lr ($\times 10^{-3}$)	0.816	0.598	0.710	0.944	0.733	0.856
l_n	5	3	4	4	6	7
l_s	32	64	64	64	64	32
bs	8	32	64	64	64	16
$\lvert\Phi\rvert$	4518	**9161**	13580	13839	22418	**7285**
Φ^l						
lr ($\times 10^{-3}$)	0.634	0.996	0.750	0.874	0.820	0.536
l_n	4	5	3	3	6	6
l_s	16	32	32	32	16	16
bs	32	64	32	32	32	8
$\lvert\Phi\rvert$	**2168**	14244	**10544**	**7900**	**9768**	11396

C. Training, Loss, and Accuracy Results

The models are trained with the hyper-parameters found according to Section V-B. The training budget is set as 2000 epochs with $nSamples = 100$ and $nIter = 25$.

Figure 3 presents the losses when learning Φ^m and Φ^l, showing that the proposed architecture improves the learning

efficiency considerably by significantly and also achieves smaller loss values which the monolithic architecture fails to achieve. Furthermore, the achieved loss difference clearly increases with more joints, indicating the benefits of having estimators with constant complexity and linking then together as in the proposed architecture.

For qualitative comparison, Figure 4 presents the joints position estimation obtained by Φ^m and Φ^l for $j = 7$ only (other joint numbers cases are omitted for briefness), showing the predictions quality difference obtained using both architectures.

Figure 5 presents the joint prediction error for 1000 samples for the robot with $j = 7$, showing that the Φ^l achieves a higher accuracy than Φ^m, which is expected due to the improved learning efficiency shown in Figure 3.

We can observe a characteristic for the models accuracies: the error increases for the further joints. For Φ^l, this results from the error propagation from one joint to another since estimation from the previous joint is used for computing the following joint's estimation; for Φ^m the increasing error is lead by the fact that the function being approximated for the latter joints is more complex (see Equation 1). It is important to emphasize that using a monolithic architecture for estimating the forward kinematics is inappropriate, as the NN outputs are related to functions with varying complexity, leading non-uniform learning in the network, which is prone to fall into local-minima. In fact, we speculate that the results presented in Figure 3 for Φ^m are explained by such local-minima: once the loss drops due to sufficient accuracy on learning the first joints, the optimization process struggles to refine the parameters, leading to the very slowly decaying loss observed after an initial sudden drop.

Next, to demonstrate the accuracy impacts, we estimate the end-effector velocity $\dot{x}_j = [\mathbf{J}\dot{q}]_j$, where $\mathbf{J}$ is the unknown Jacobian matrix of the robot, as $\tilde{\dot{x}}_j = \left[\frac{\partial \Phi}{\partial q}\dot{q}\right]_j$. That is, we use the learned network for estimating the unknown Jacobian of the model, which has its well-known applications in inverse kinematics [30] computation, Cartesian impedance controllers [31], and tracking controllers [32]. Figure 6 shows the velocity estimation and ground-truth obtained from the simulator for $n = 3$ and $j = 7$. The better accuracy of the proposed architecture indicates its adequacy for being used in higher level applications, while the monolithic architecture poor accuracy might impact higher level applications badly.

VI. Conclusions

This paper presents a novel NN architecture for learning the forward kinematics of serial robot manipulators. Instead of a monolithic approach in which the joints positions are estimated with a relatively large NN, the proposed architecture leverages the DH-Transformation notation, resulting in a link-by-link estimator and keeping a constant complexity for the estimation between links.

Results show that the proposed architecture improves the learning efficiency, regarding to the number of samples, and achieves better precision when compared to the traditional monolithic approach found in the literature. Furthermore, the proposed architecture is shown scale better for robots with increasing number of joints.

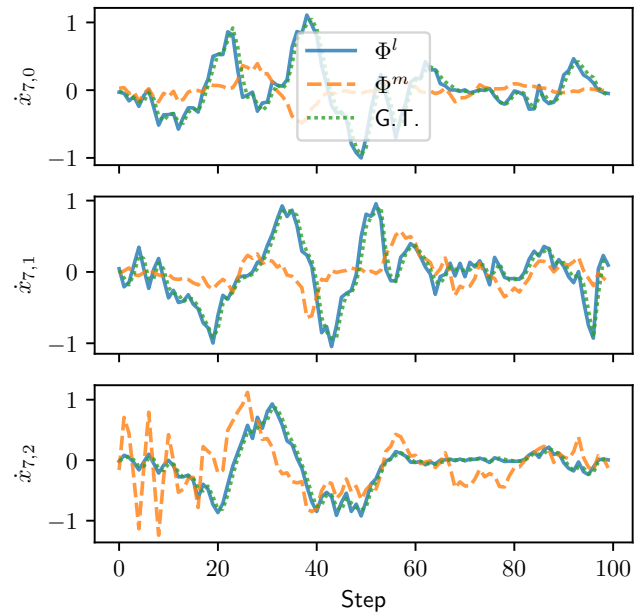

Fig. 6. End-effector velocity estimation comparison for Φ^m and Φ^l.

Acknowledgements

This work has been supported by the PNRR MUR project PE0000013-FAIR

References

[1] A. Albu-Schaffer and G. Hirzinger, "Cartesian impedance control techniques for torque controlled light-weight robots," in *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292)*, vol. 1, 2002, pp. 657–663 vol.1.

[2] S. Scherzinger, A. Roennau, and R. Dillmann, "Forward dynamics compliance control (fdcc): A new approach to cartesian compliance for robotic manipulators," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 4568–4575.

[3] N. Das and M. C. Yip, "Forward kinematics kernel for improved proxy collision checking," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2349–2356, 2020.

[4] L. Čehovin Zajc, A. Rezelj, and D. Skočaj, "Low-Cost Open-Source Robotic Platform for Education," *IEEE Transactions on Learning Technologies*, vol. 16, no. 1, pp. 18–25, 2023.

[5] L. Nguyen, R. Patel, and K. Khorasani, "Neural network architectures for the forward kinematics problem in robotics," in *1990 IJCNN International Joint Conference on Neural Networks*, 1990, pp. 393–399 vol.3.

[6] M. Theofanidis, S. I. Sayed, J. Cloud, J. Brady, and F. Makedon, "Kinematic Estimation with Neural Networks for Robotic Manipulators," in *Artificial Neural Networks and Machine Learning – ICANN 2018*, V. Kůrková, Y. Manolopoulos, B. Hammer, L. Iliadis, and I. Maglogiannis, Eds. Cham: Springer International Publishing, 2018, pp. 795–802.

[7] K. P. and B. B. Choudhury, "Transfer Learning-Based Artificial Neural Network for Forward Kinematic Estimation of 6-DOF Robot," *International Journal of Applied Metaheuristic Computing (IJAMC)*, vol. 13, no. 1, pp. 1–15, January 2022.

[8] P. Kesaba and B. Choudhury, "Forward Kinematic Analysis of 6-R Industrial Robot Using Active Sampled Neural Network Regressor," *International Journal of Software Innovation*, vol. 10, no. 1, pp. 1–14, 2022 2022.

[9] A. Dalla Libera, N. Castaman, S. Ghidoni, and R. Carli, "Autonomous Learning of the Robot Kinematic Model," *IEEE Transactions on Robotics*, vol. 37, no. 3, pp. 877–892, 2021.

[10] A.-N. Sharkawy, "Forward and inverse kinematics solution of a robotic manipulator using a multilayer feedforward neural network," *Journal of Mechanical and Energy Engineering*, vol. 6, no. 2, Oct. 2022.

[11] A.-N. Sharkawy and S. Khairullah, "Forward and Inverse Kinematics Solution of A 3-DOF Articulated Robotic Manipulator Using Artificial Neural Network," *International Journal of Robotics and Control Systems*, vol. 3, no. 2, pp. 330–353, 2023.

[12] B. Damas and J. Santos-Victor, "An online algorithm for simultaneously learning forward and inverse kinematics," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 1499–1506.

[13] H. Ren and P. Ben-Tzvi, "Learning inverse kinematics and dynamics of a robotic manipulator using generative adversarial networks," *Robotics and Autonomous Systems*, vol. 124, p. 103386, 2020.

[14] I. M. Loutfi, A. B. Boutchouang, A. Melingui, O. Lakhal, F. B. Motto, and R. Merzouki, "Learning-Based Approaches for Forward Kinematic Modeling of Continuum Manipulators," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9899–9904, 2020, 21st IFAC World Congress.

[15] A. Morell, M. Tarokh, and L. Acosta, "Solving the forward kinematics problem in parallel robots using Support Vector Regression," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 7, pp. 1698–1706, 2013.

[16] S. T. Kanneganti, J. Pei, and D. F. Hougen, "Developing interpretable machine learning for forward kinematics of robotic arms," in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2021, pp. 01–09.

[17] F. Cursi, W. Bai, W. Li, E. M. Yeatman, and P. Kormushev, "Augmented neural network for full robot kinematic modelling in se(3)," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7140–7147, 2022.

[18] R. M. Grassmann and J. Burgner-Kahrs, "On the Merits of Joint Space and Orientation Representations in Learning the Forward Kinematics in SE(3)," in *Proceedings of Robotics: Science and Systems*, FreiburgimBreisgau, Germany, June 2019.

[19] R. Bouzid, J. Narayan, and H. Gritli, "Exploring neural networks for forward kinematics of the robotic arm with different length configurations: A comparative analysis," in *2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, vol. 2, 2024, pp. 1–6.

[20] D. Kubus, R. Rayyes, and J. J. Steil, "Learning Forward and Inverse Kinematics Maps Efficiently," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 5133–5140.

[21] F. Gonçalves, T. Ribeiro, A. F. Ribeiro, G. Lopes, and P. Flores, "A recursive algorithm for the forward kinematic analysis of robotic systems using euler angles," *Robotics*, vol. 11, no. 1, 2022.

[22] S. Kucuk and Z. Bingul, *Robot Kinematics: Forward and Inverse Kinematics*, S. Cubero, Ed. InTech, 2006.

[23] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," 2016.

[24] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.

[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035.

[26] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in PyTorch," in *NIPS-W*, 2017.

[27] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, "Tune: A Research Platform for Distributed Model Selection and Training," *arXiv preprint arXiv:1807.05118*, 2018.

[28] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[29] L. Li, K. Jamieson, A. Rostamizadeh, K. Gonina, M. Hardt, B. Recht, and A. Talwalkar, "Massively Parallel Hyperparameter Tuning," 2018.

[30] K. J. Waldron, S.-L. Wang, and S. J. Bolin, "A Study of the Jacobian Matrix of Serial Manipulators," *Journal of Mechanisms, Transmissions, and Automation in Design*, vol. 107, no. 2, pp. 230–237, 06 1985.

[31] P. Sánchez-Sánchez and F. Reyes-Cortés, "Cartesian control for robot manipulators," *robot manipulators trends and development*, pp. 165–212, 2010.

[32] D. Chen, Y. Zhang, and S. Li, "Tracking Control of Robot Manipulators with Unknown Models: A Jacobian-Matrix-Adaption Method," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 7, pp. 3044–3053, 2018.